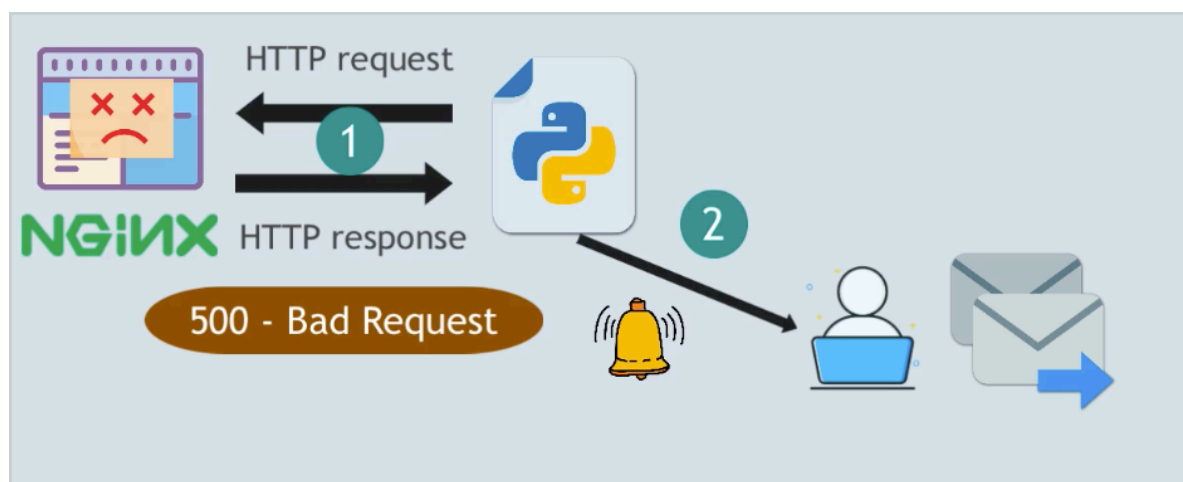


Module 14 - chapter 14

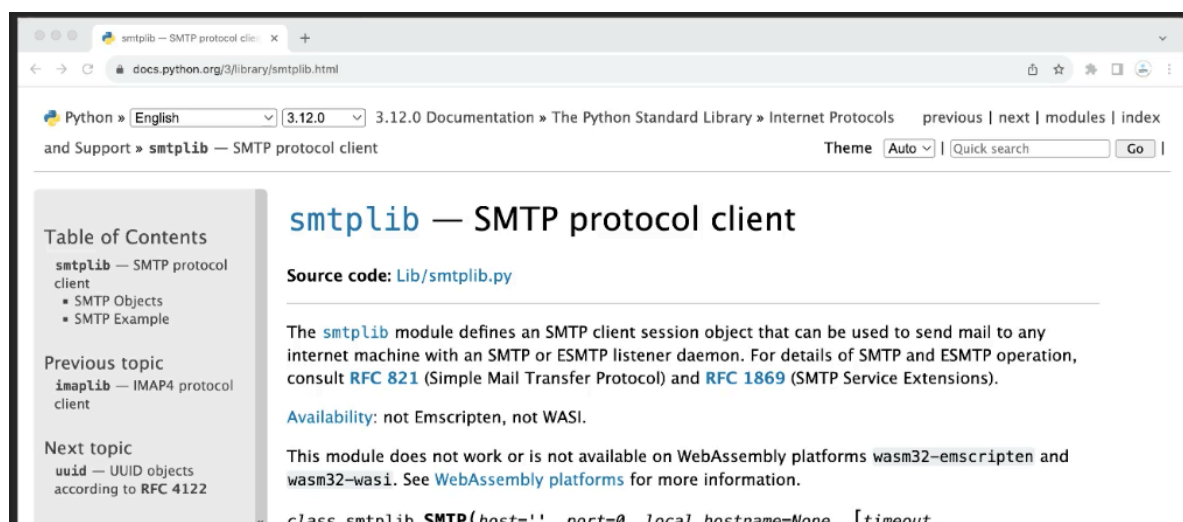
Website Monitoring 2: Automated Email Notification

Email Notification

Following from the previous demo, we check whether the app is up and running or is down. Now as part of monitoring this is not enough we don't want to check this ad hoc or manually, we want that when the app is down it gets notify to somebody so the problem is fixed and for that we will send an email to notify the app is down.

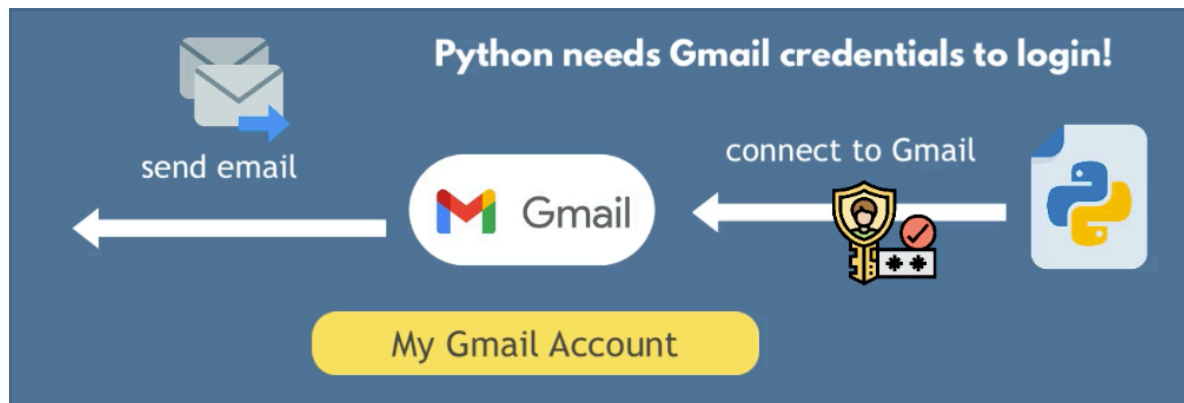


To send emails python has an in-built library called -> smtplib



And we need a sender and a receiver, so from one email account to another email.

So python needs to be able to access the email account and send an email from our behalf.



So python needs credentials to access the email account.

Configure your email provider

-> `smtplib.SMTP("smtp.gmail.com", 587)`

"smtp.gmail.com" -> host, name of the email provider

587 -> smtp port for gmail (email provider)

Incoming Mail (IMAP) Server <pre> # send email to me smtplib.SMTP('smtp.gmail.com', 587) </pre>	imap.gmail.com Requires SSL: Yes Port: 993
Outgoing Mail (SMTP) Server	smtp.gmail.com Requires SSL: Yes Requires TLS: Yes (if available) Requires Authentication: Yes Port for SSL: 465 Port for TLS/STARTTLS: 587

Here is google documentation link <https://knowledge.workspace.google.com/admin/gmail/send-email-from-a-printer-scanner-or-app>

Considerations

When using the Gmail SMTP server:

- The sending limit is 2,000 messages per day. Learn more about [email sending limits](#).
- Spam filters might reject or filter suspicious messages.
- The fully qualified domain name of the SMTP service is `smtp.gmail.com`.
- Configuration options include:
 - Port 25, 465, or 587
 - [SSL and TLS protocols](#)
 - Dynamic IP addresses

And we use 'with' statement which is a python syntax to deal with resources that are not inside our control (external resources, applications, files) and it will handle issues that might occur regarding those resources.

The image is a composite graphic comparing two ways to handle files in Python. On the left, under the heading 'without using with statement:', two code snippets are shown. The first snippet uses `open()`, `write()`, and `close()` directly. The second snippet uses a `try:` block for `write()` and a `finally:` block for `close()`. On the right, under the heading 'using with statement:', a code snippet shows the `with open() as file:` syntax. To the right of this is a blue box titled 'with statement' with a lightbulb icon, listing its benefits: it's an alternative to try/finally, used for exception handling and cleanup to make code cleaner, and used with unmanaged resources like file handling. The bottom right corner features a logo for 'TECHWORLD WITH NANA'.

without using with statement:

```
# 1) without using with statement
file = open('file_path', 'w')
file.write('hello world !')
file.close()

# 2) without using with statement
file = open('file_path', 'w')
try:
    file.write('hello world')
finally:
    file.close()
```

using with statement:

```
# using with statement
with open('file_path', 'w') as file:
    file.write('hello world !')
```

with statement

- alternative to try/finally statements
- is used in exception handling and clean up code to make code cleaner
- used with unmanaged resources (e.g. file handling)

TECHWORLD WITH NANA

So if there is an issue with the email provider in this DEMO, then 'with' will deal with it and it will clean up all the resources for us in the background. like in the screenshot above we don't have to deal with `.close()`, that will be taken care for us.

And this is what we have so far:

```
monitor-website.py x
1 import requests
2 import smtplib
3
4 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:80
5
6 if response.status_code == 200:
7     print("Application is running successfully!")
8 else:
9     print("Application Down. Fix it!")
10    # send notification email to support
11    with smtplib.SMTP( host: "smtp.gmail.com", port: 587) as smtp: You, Mome
12
```

So now we have a variable that we named 'smtp' and we can use it to call methods.

We need to encrypt our message from python to the email service. So the communication is secure so we need to start TLS

-> ,starttls()

And I take a peak

```
Automation_with_python v Module_14/Automate_Python_chapter-14 v monitor-website v
monitor-website.py smtpplib.py x
197 class SMTP:
696 def login(self, user: str, password: str, *, initial_response_ok: bool = True):
761     # We could not login successfully. Return result of last attempt.
762     raise last_exception
763
764 * def starttls(self, *, context: SSLContext | None = None):
765     """Puts the connection to the SMTP server into TLS mode.
766
767     If there has been no previous EHLO or HELO command this session, this
768     method tries ESMTP EHLO first.
769
770     If the server supports TLS, this will encrypt the rest of the SMTP
771     session. If you provide the context parameter,
772     the identity of the SMTP server and client can be checked. This,
773     however, depends on whether the socket module really checks the
774     certificates.
775
776     This method may raise the following exceptions:
777
778     SMTPHeloError The server didn't reply properly to
779     the helo greeting.
780     """
```

And I have a question for AI:

Q:

I am reading .starttls() method. and it says:"If the server supports TLS, this will encrypt the rest of the SMTP session." so I want to check if by server it means the email provider and that the sentence is saying if the server provider for

example gmail supports TLS then the rest of the session will be encrypted. is this correct?

A:

Yes, that's correct. "The server" refers to whichever SMTP server you've connected to in that session — e.g., if you did `smtpplib.SMTP('smtp.gmail.com', 587)`, then "the server" is Gmail's SMTP server.

A bit more detail on what's happening:

1. `smtpplib.SMTP(host, port)` opens a **plain-text** TCP connection to that server.
2. `.starttls()` sends the STARTTLS command, which **upgrades** that same connection to **TLS-encrypted** — it doesn't open a new connection.
3. Everything after that point (login credentials, MAIL FROM, RCPT TO, the message body) travels encrypted instead of in plain text.

One nuance: STARTTLS only encrypts your connection to that SMTP server — the hop from your machine to Gmail (or whatever provider you're relaying through). If the recipient is on a different mail server, what happens between Gmail and the recipient's server is a separate hop, governed by whatever those two servers negotiate — you have no control over it from your script.

Also worth knowing: if the server doesn't support STARTTLS, calling `.starttls()` raises `SMTPNotSupportedError` rather than silently sending unencrypted — so it's a hard failure, not a silent fallback.

```
11         with smtpplib.SMTP( host: "smtp.gmail.com", port: 587) as smtp:
12             smtp.starttls()
13
```

Next we call `-> .ehlo()`

To identify our python application with the email server on this encrypted communication.

```
11         with smtpplib.SMTP( host: "smtp.gmail.com", port: 587) as smtp:
12             smtp.starttls()
13             smtp.ehlo()
```

And I have another question for AI:

Q:

We are using this code: `with smtpplib.SMTP("smtp.gmail.com", 587) as smtp:`
`smtp.starttls()`
`smtp.ehlo()`

but `.starttls()` says that "If there has been no previous EHLO or HELO command this session, this method tries ESMTP EHLO first." so I wonder if `.ehlo()` is redundant?

A:

No — they serve different purposes and neither is redundant. Here's what's actually happening in that code:

1. `smtp.starttls()` — since no EHLO/HELO has been sent yet, it silently sends one for you first, in plain text, to check the server's capabilities and confirm STARTTLS is offered. Then it does the actual TLS handshake.
2. `smtp.ehlo()` — after the handshake, this sends a second EHLO, now over the encrypted channel.

That second, explicit EHLO matters because:

- Trust: the first EHLO's response arrived unencrypted, so it could in theory be tampered with by a man-in-the-middle before TLS kicks in. The post-TLS EHLO gets you a response you can actually trust.
- Capabilities can differ under TLS: many servers (Gmail included) only advertise certain AUTH mechanisms once the connection is encrypted, since sending credentials in the clear would be unsafe. If you skip the second EHLO, `smtp.esmtp_features` may not reflect what auth methods are actually available, and `.login()` could fail or pick a weaker mechanism.

So the pattern is: implicit pre-TLS EHLO (just to check STARTTLS support) → STARTTLS handshake → explicit post-TLS EHLO (to get the real, trustworthy capability list before authenticating).

q:

how a `ehlo/helo` inside `starttls()` checks the server capabilities? what are capabilities, are they the functions available like `starttls()`?

A:

let's trace what `starttls()` actually does internally (this is Python's real `smtplib` source, simplified):

```
def starttls(self, ...):
    self.ehlo_or_helo_if_needed()      # step 1
    if not self.has_extn("starttls"):  # step 2
        raise SMTPNotSupportedError(...)
    (resp, reply) = self.docmd("STARTTLS") # step 3
    ...
```

Step 1 — `ehlo_or_helo_if_needed()`:

```
def ehlo_or_helo_if_needed(self):
```

```
if self.helo_resp is None and self.ehlo_resp is None:
    self.ehlo() # (falls back to helo() if ehlo fails)
```

It checks: "have I already sent EHLO/HELO this session?" If not, it calls `self.ehlo()` right there, which sends the literal EHLO <hostname> command over the socket and reads back the server's multi-line 250-... reply

Here's roughly what happens on the wire when you call `self.ehlo()`:

```
C: EHLO mymachine.local
S: 250-smtp.gmail.com at your service, [1.2.3.4]
S: 250-SIZE 35882577
S: 250-8BITMIME
S: 250-STARTTLS
S: 250-ENHANCEDSTATUSCODES
S: 250-PIPELINING
S: 250-CHUNKING
S: 250 SMTPUTF8
```

`self.ehlo()` then parses that text response into a dict, roughly:

```
self.esmtp_features = {
    "size": "35882577",
    "8bitmime": "",
    "starttls": "",
    "auth": "LOGIN PLAIN XOAUTH2",
    ...
}
```

Each 250-KEYWORD ... line becomes one entry, keyed by the lowercased keyword.

Step 2 — `has_extn("starttls")` just does "starttls" in `self.esmtp_features` — a dict lookup against what was parsed in step 1. Nothing more clever than that.

Step 3 — only if that lookup succeeded does it actually send the STARTTLS command.

So "checking the server's capabilities" = send EHLO → read the reply → parse each 250- line into a dict → look up a key in that dict. It's string parsing of a response the server sent you, not some separate handshake.

To your capabilities-vs-functions question directly: a capability is one line/ keyword the server chose to include in its EHLO reply (STARTTLS, AUTH, SIZE, 8BITMIME...) — it's the server's own self-reported feature list, defined by whichever RFC introduced that extension. `smtp.starttls()` is a Python method

that (a) checks whether "starttls" is present in that dict, and (b) if so, sends the corresponding SMTP command. The method exists in smtplib regardless of the server; whether calling it will succeed depends entirely on whether that capability showed up in the EHLO reply.

So we have a secure connection from python to gmail account And then we can do .login()

```
11         with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
12             smtp.starttls()
13             smtp.ehlo()
14             smtp.login(user="", password="")
```

Because my google account has the 2-step verification ON (2 factor authentication or MFA - multi factor authentication) then I am good to go.

But if I didn't have that then I need to edit my gmail account to allow calls from applications like the script we are writing.

```
smtp.login("nana@techworld-with-nana.com", "")
```

without two-way verification

- provide email and password in python code
- enable "less secure apps" in your Google account

And this needs to be allowed -> less secure app access

myaccount.google.com/lesssecureapps?pli=1&rapt=AEjHL4PMY4Fy8xHCUG9-10ckWBcQN6asKB-7rusEITXl-1cm-rjvPL8l60hkFdYaz_wAJETLIIRyS9C-Sx-xFZuk-PYkpnCcZ40BveuZB36xg_7ID-3Qs

Google Account

← Less secure app access

Some apps and devices use less secure sign-in technology, which makes your account vulnerable. You can turn off access for these apps, which we recommend, or turn it on if you want to use them despite the risks. Google will automatically turn this setting OFF if it's not being used. [Find out more](#)

Allow less secure apps: OFF

In my case because I have MFA then less secure apps access is not available to me, but also since Jan 2025 this is not an option for anyone:

Less secure apps & your Google Account

Starting January 2025, less secure apps, third-party apps, or devices that have you sign in with only your username and password will no longer be supported for Google Workspace accounts. For exact dates, visit [Google Workspace Updates](#). To continue to use a specific app with your Google Account, you'll need to use a more secure type of access that doesn't share password data. [Learn how to use Sign in with Google](#).

If an app or site doesn't meet our [security standards](#), Google might block anyone who's trying to sign in to your account from it. Less secure apps can make it easier for hackers to get in to your account, so blocking sign-ins from these apps helps keep your account safe.

Use more secure apps

If an app uses less secure sign-in technology, you might not be able to use it with your Google Account.

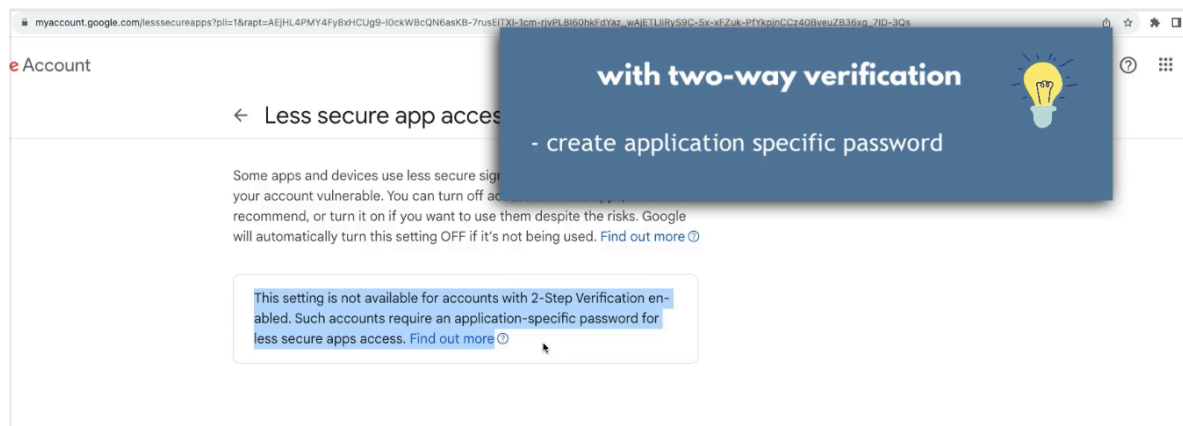
Apps that use only your username or password will create errors if they try to access your Google Account. These errors can appear as:

- "Invalid username"
- "Invalid password"
- "Unable to log in"

You can sign in to your Google Account from any app that has the "Sign in with Google" option. In some cases you may need to remove your Google Account and then add it again on your device.

 Give feedback about this article

So I need to create a password for the specific application:



myaccount.google.com/lesssecureapps?pli=1&rapto=AEJHL4PMY4Fy8xHCUG9-l0ckWBcQ6asKB-7rusEITXl-1cm-rvPLB60hkFdYaz_wAJETLIRyS9C-Sx-xFZuk-PTYxpnCCZ40BveuZB36va_7ID-3Qs

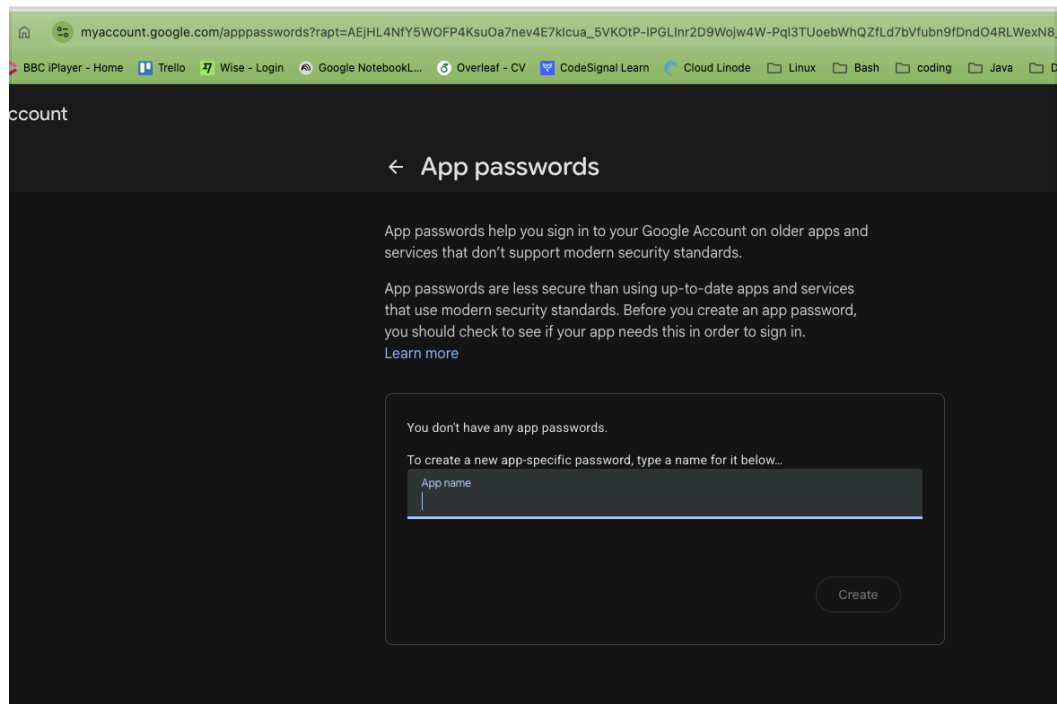
Account

← Less secure app access

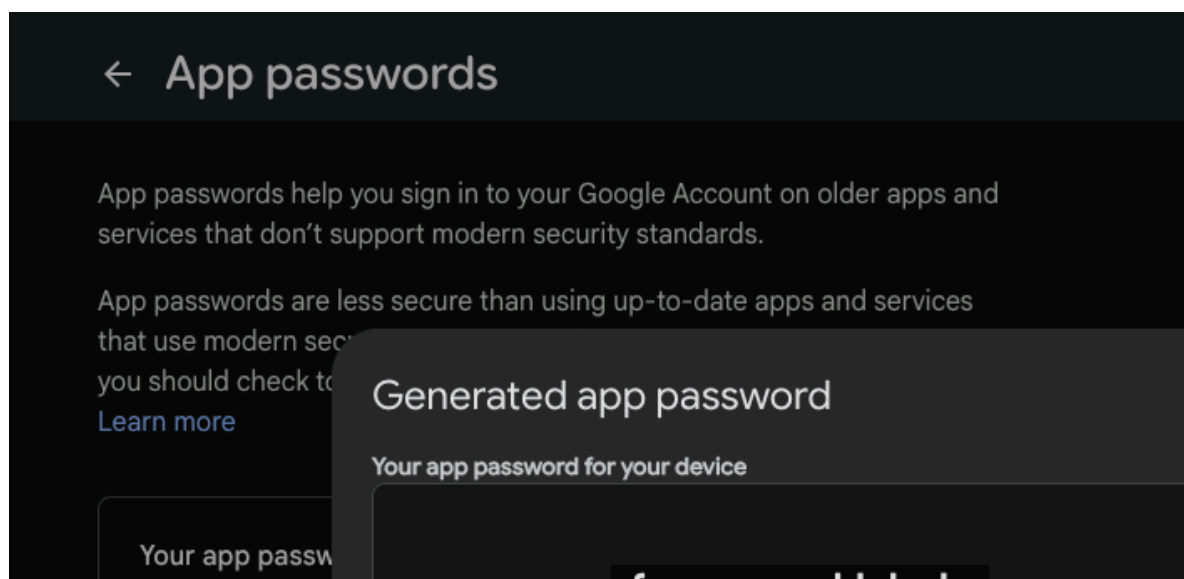
with two-way verification
- create application specific password

Some apps and devices use less secure sign-in technology that makes your account vulnerable. You can turn off access to these apps, which we recommend, or turn it on if you want to use them despite the risks. Google will automatically turn this setting OFF if it's not being used. [Find out more](#)

This setting is not available for accounts with 2-Step Verification enabled. Such accounts require an application-specific password for less secure apps access. [Find out more](#)



I give it a name -> 'website-monitor' and then I get a password



How to use it

Go to the settings for your Google Account in the application or device you are trying to set up. Replace your password with the 16-character password shown above.

Just like your normal password, this app password grants complete access to your Google Account. You won't need to remember it, so don't write it down or share it with anyone.

Done

Q:

i created a password for my gmail account using apppasswords, the password has empty spaces, I wonder if this is part of the password or when I need to use it I remove the spaces?

A:

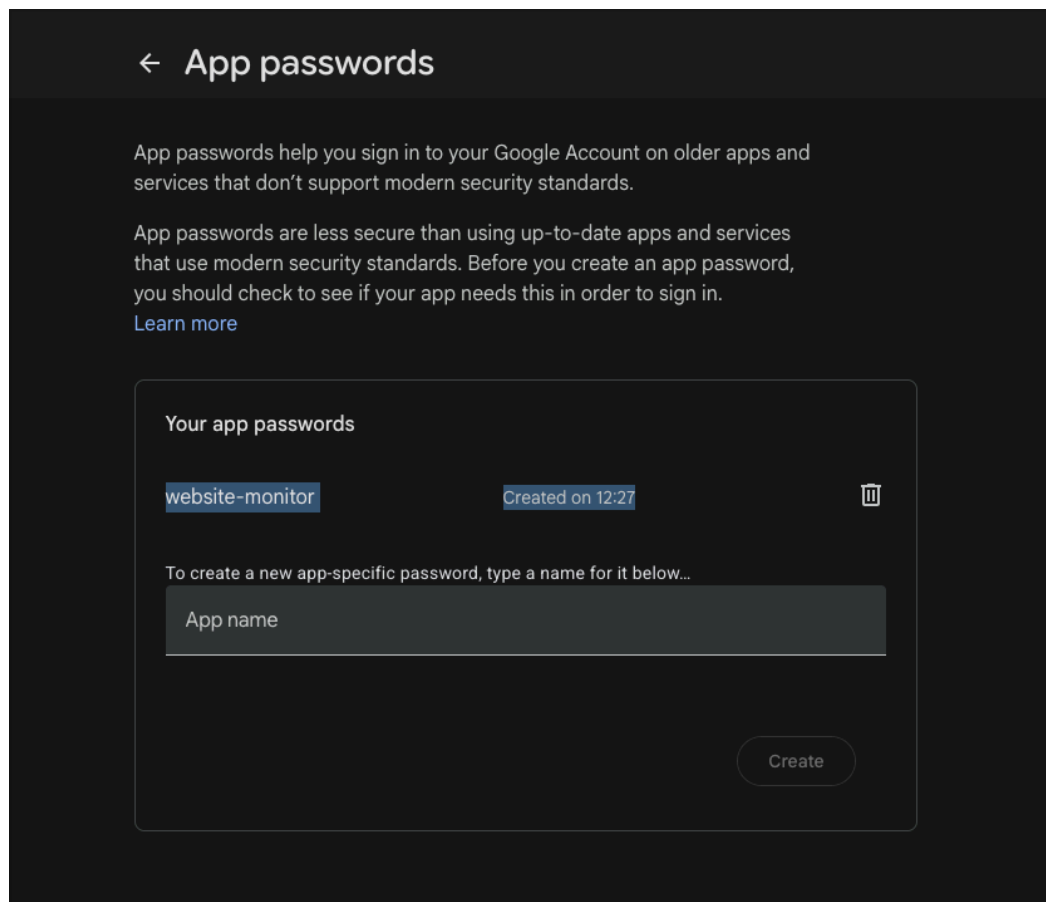
Keep the spaces when Google displays it (they're just for readability, grouped in 4-character chunks), but when you actually use it in code, you can include or remove them — both work, because Gmail's auth strips whitespace from the password before checking it.

So in practice:

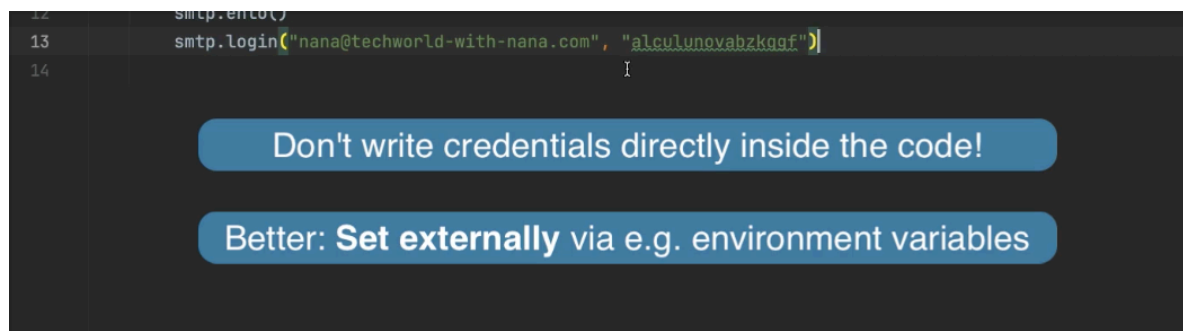
```
smtp.login("you@gmail.com", "abcd efgh ijkl mnop")  # works  
smtp.login("you@gmail.com", "abcdefghijklmnop")    # also works
```

Most people just paste it without spaces since it's a single token conceptually — but if you paste it with spaces intact, that's fine too. What would break it is typos or dropping/adding characters, so copy-paste rather than retyping either way.

And we can see the new password listed in App passwords:



And we still need to improve the code as for security reasons is not good practice to hardcode the passwords in the application.



So we either set the credentials in:

- Local file -> config file so python can access it.
- Environment variables and then access them.

We will set env vars using library OS

```
monitor-website.py x
1 import requests
2 import smtplib
3 import os
4
5 os.environ
6
7
8 response = requests.get('http://172.10.4.1:8080')
9 if response.status_code == 200:
10     print('Application is running successfully')
11 else:
12     print('Application Down. Fix it!')
13     # send email to me
14     with smtplib.SMTP('smtp.gmail.com', 587) as smtp:
15         smtp.starttls()
16         smtp.ehlo()
17         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
```

os module
- provides operating system dependent functionality

And now we can get the env vars:

```
2 import smtplib
3 import os
4
5 EMAIL_ADDRESS = os.environ.get('EMAIL_ADDRESS')
6 EMAIL_PASSWORD = os.environ.get('EMAIL_PASSWORD')
7
8 # "nanahworld-with-nana.com", "alcyon"
9
10
11 response = requests.get('http://172.10.4.1:8080')
12 if response.status_code == 200:
13     print('Application is running successfully')
14 else:
15     print('Application Down. Fix it!')
16     # send email to me
17     with smtplib.SMTP('smtp.gmail.com', 587) as smtp:
18         smtp.starttls()
19         smtp.ehlo()
20         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
```

Constants
- Best practice: to write them in uppercase
- In Python: there is NO difference, just semantics
Python interpreter treats them the same

This is the code so far:

```

1  import requests
2  import smtplib
3  import os
4
5  EMAIL_ADDRESS = os.environ.get("EMAIL_ADDRESS")
6  EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
7
8  response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
9
10 if response.status_code == 200:
11     print("Application is running successfully!")
12 else:
13     print("Application Down. Fix it!")
14     # send notification email to support
15     with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
16         smtp.starttls()
17         smtp.ehlo()
18         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
19

```

Now we need to set the env vars

If we use 'export'

```

nana@macbook /Users/nana
% export EMAIL_ADDRESS="nana@techworld-with-nana.com"
nana@macbook /Users/nana
% echo $EMAIL_ADDRESS
nana@techworld-with-nana.com
nana@macbook /Users/nana
%

```

The env var is only available for this terminal session, so if we open another terminal the env vars are not there, same if I close the terminal tab then the env var is gone.

We need the env vars to be global and persistent _ so we set the env vars in the OS.

For Mac there is a file `.bash_profile` which is a config file that Bash runs every time a new Bash session is created.

Bash Profile



- *bash_profile* is a config file that Bash runs every time a new Bash session is created

But I use zsh so the config file I need to use is .zshrc !!

Q:

For Mac there is a file .bash_profile which is a config file that Bash runs every time a new Bash session is created. a zsh is still a bash session?

A:

No — zsh and bash are two separate, independent shell programs (different interpreters), not variants of the same thing. zsh does not read .bash_profile at all by default.

Since macOS Catalina (2019), zsh is the default shell on Mac (your current shell, per this session, is zsh). Each shell has its own config files it looks for:

bash reads (for login shells): .bash_profile, or .bash_login, or .profile (first one found)

zsh reads instead:

- .zshenv — always, every session
- .zprofile — login shells only
- .zshrc — interactive shells only (this is the one people usually edit, analogous to .bashrc)
- .zlogin — login shells, after .zshrc
- .zlogout — when a login shell exits

So if you have stuff in .bash_profile (e.g., PATH exports, aliases) and you're now on zsh, none of it runs automatically — that's a common gotcha when people switch from bash to zsh, or move an old dotfiles setup to a newer Mac. To carry settings over, you'd either recreate them in .zshrc/.zprofile, or explicitly source the old file, e.g. add `source ~/.bash_profile` to .zprofile.

You can check which shell you're actually running with `echo $SHELL` or `echo $0`.

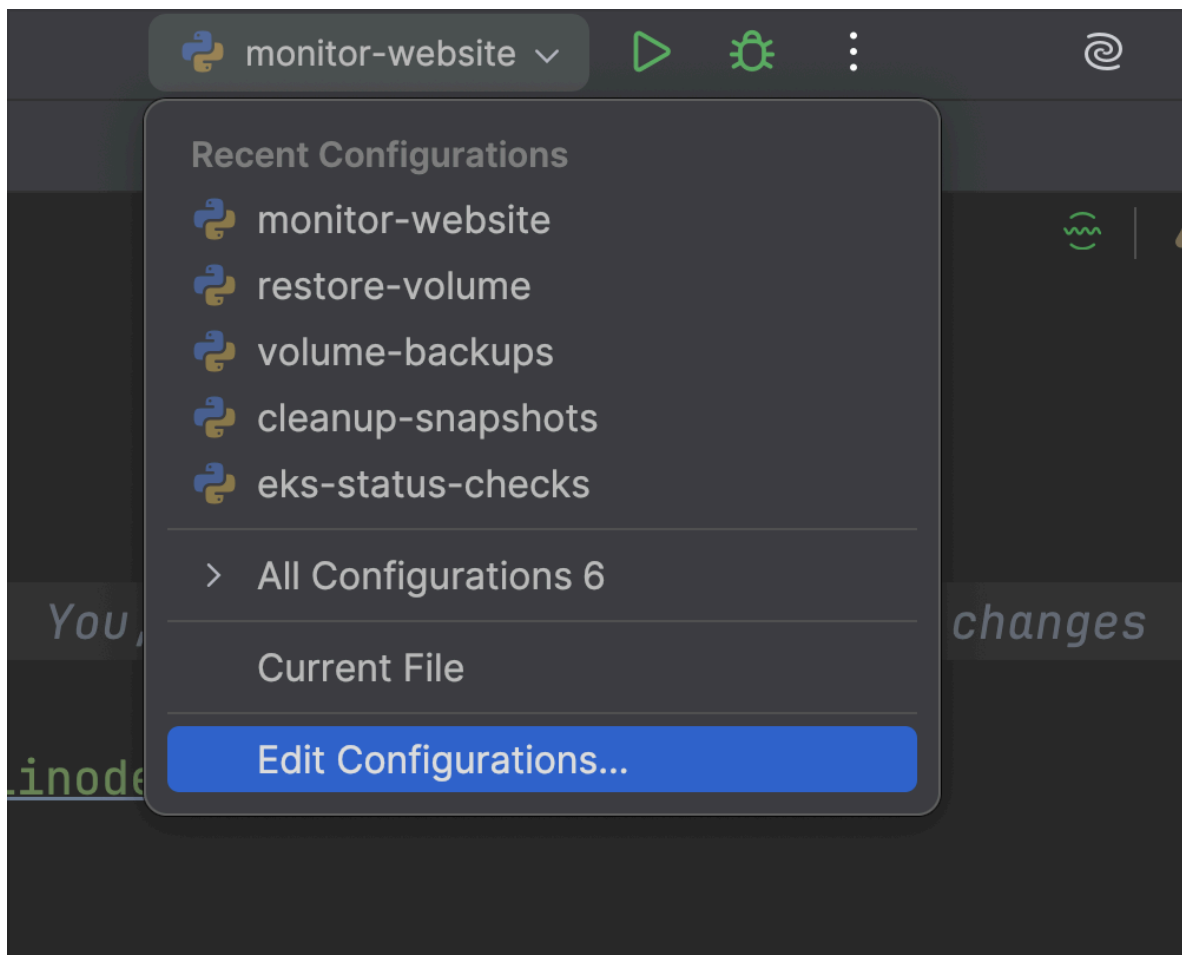
So in my case

```
~ cat ~/.zshrc
```

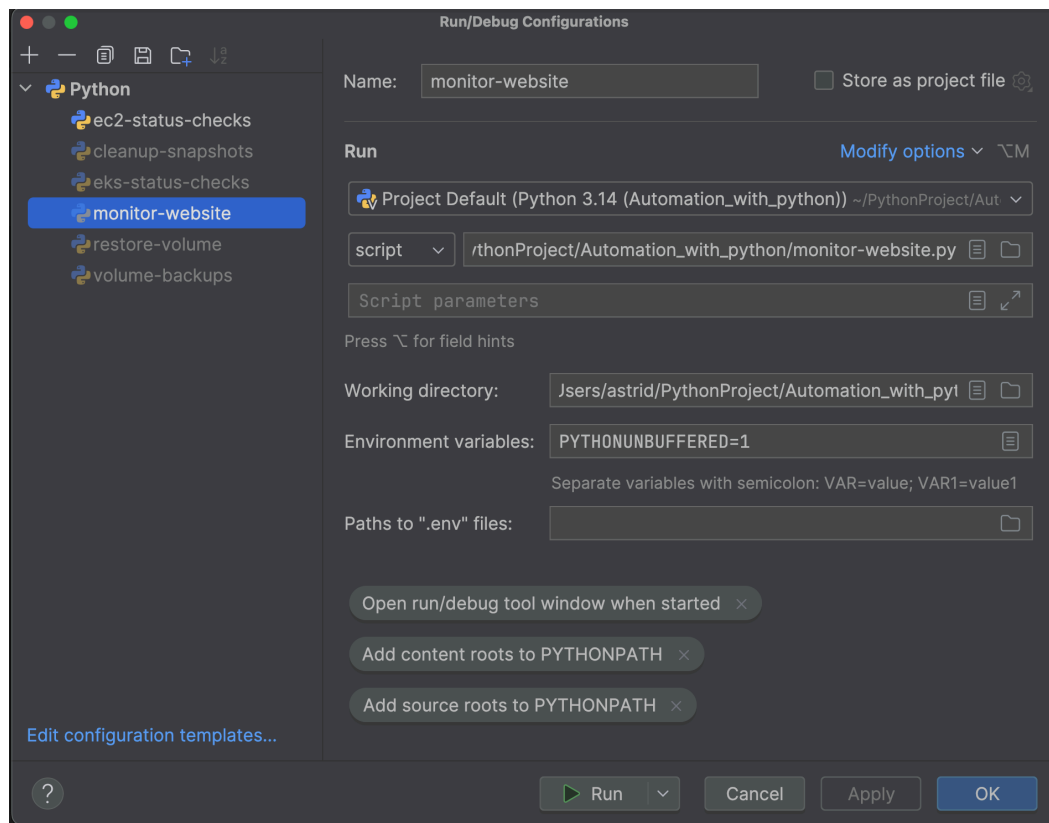
But if I edit this file then I have to restart the laptop for the terminal sessions to now be aware of this value.

So as an alternative we can use Pycharm editor to set the env vars so we don't have to restart the machine and in this case the Linode server, and the env vars will be only for this python project.

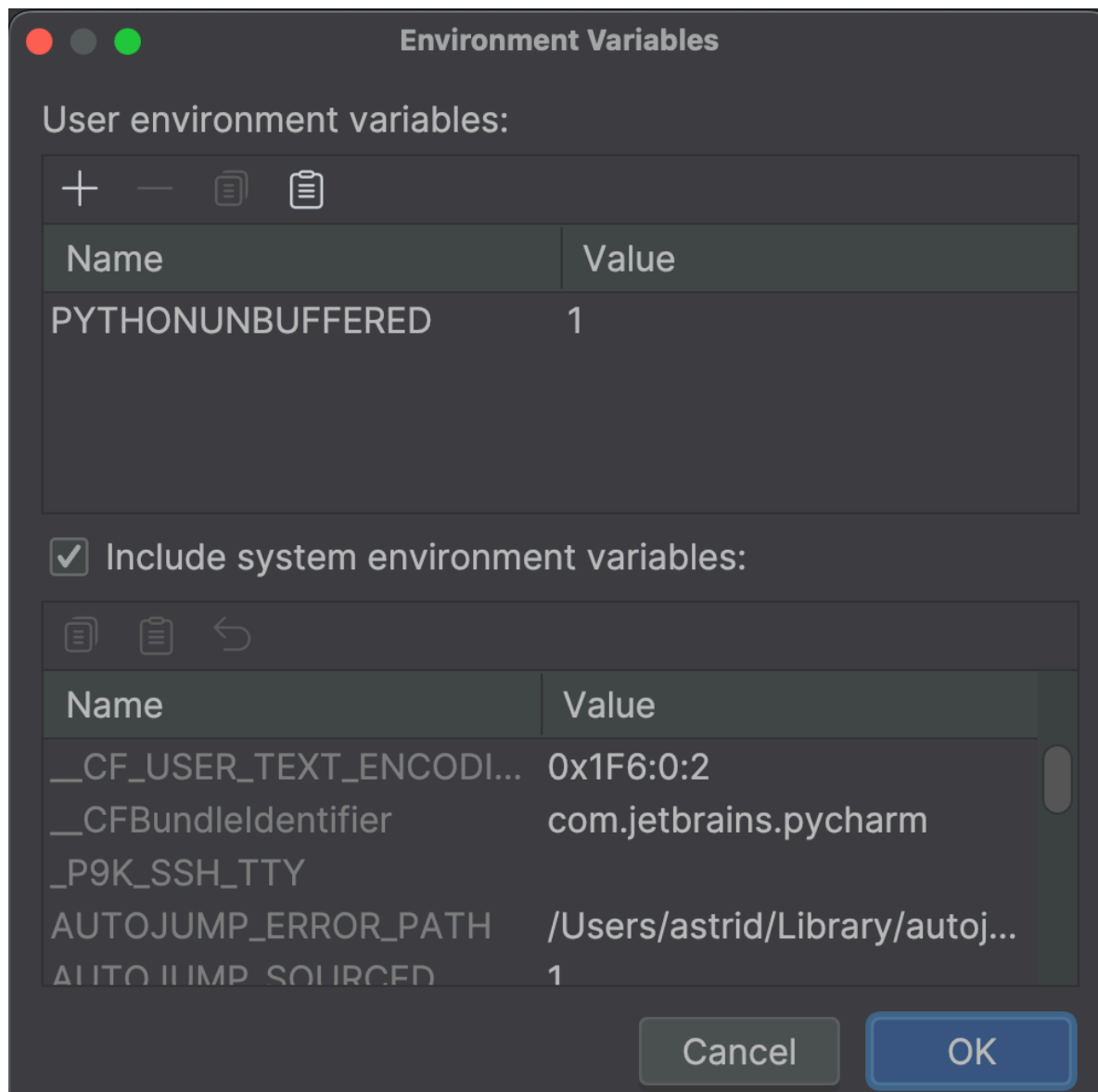
So we go to Edit Configurations



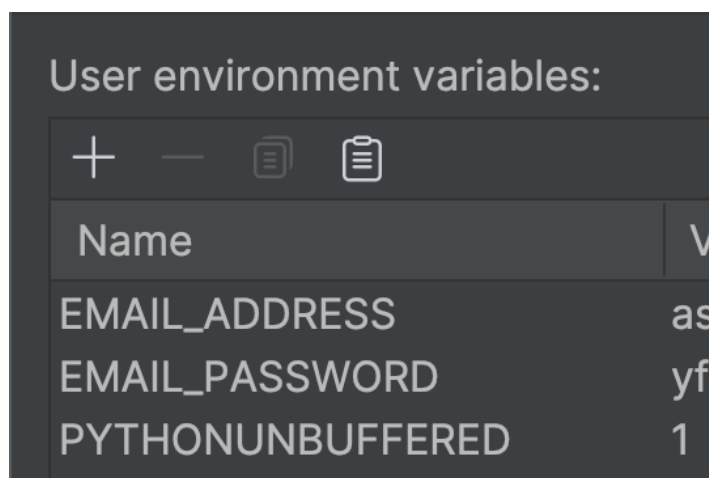
Then we select environment variables



And we see we have one by default

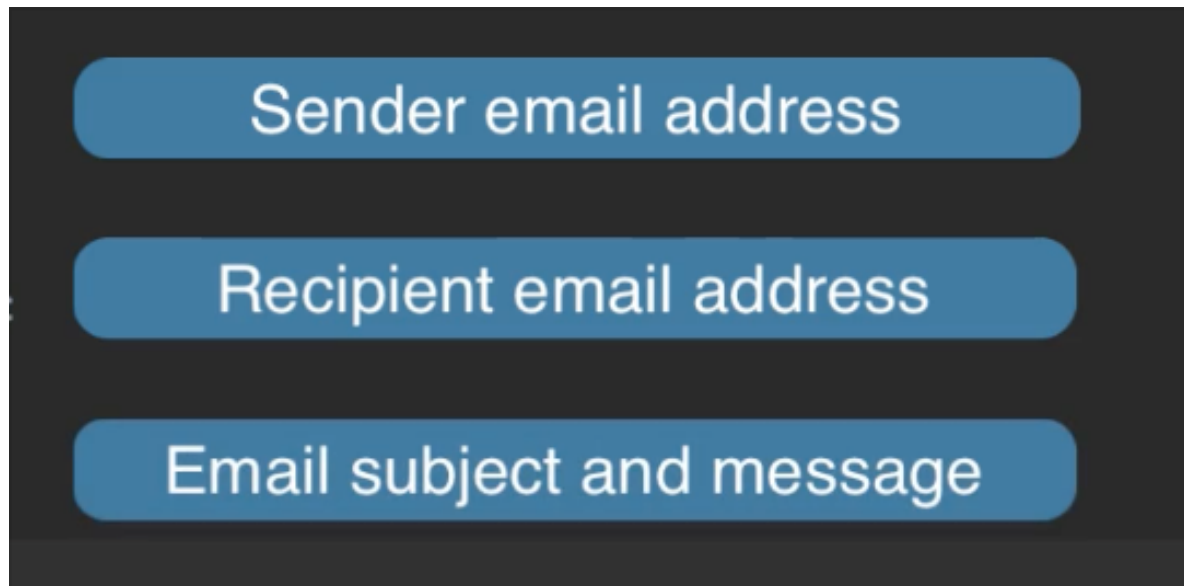


And we add our env vars:



Now we can send an email
-> .sendmail()

We need to pass 3 arguments:



And now we get this code:

```
3 import os
4
5 EMAIL_ADDRESS = os.environ.get("EMAIL_ADDRESS")
6 EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
7
8 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
9
10 if response.status_code == 200:
11     print("Application is running successfully!")
12 else:
13     print("Application Down. Fix it!")
14     # send notification email to support
15     with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
16         smtp.starttls()
17         smtp.ehlo()
18         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
19         msg = "Subject: SITE DOWN\nFix the issue! Restart the application."
20         smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
21
22
```

Now let's test the script:

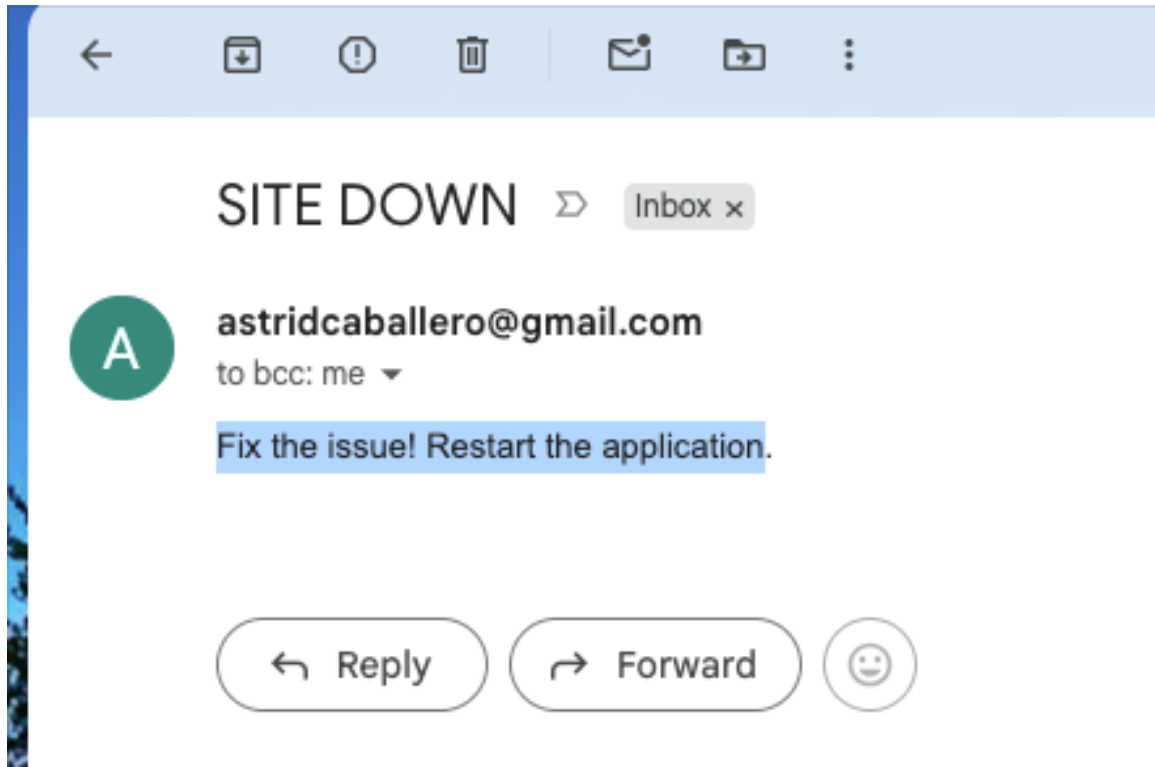
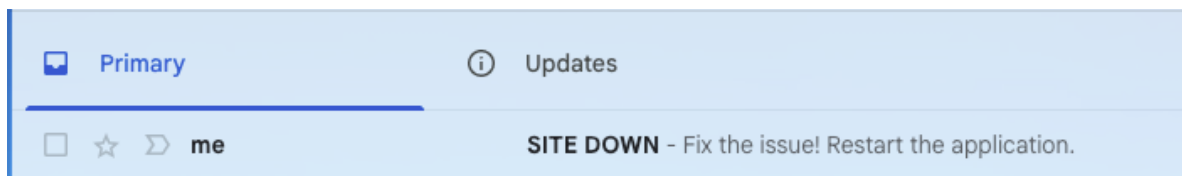
```
monitor-website.py x
6 EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
7
8 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
9
10 if response.status_code == 200:
11     print("Application is running successfully!")
12 else:
13     print("Application Down. Fix it!")
14     # send notification email to support
15     with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
16         smtp.starttls()
17         smtp.ehlo()
18         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
19         msg = "Subject: SITE DOWN\nFix the issue! Restart the application."
20         smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
21
Run monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Py
Application is running successfully!
Process finished with exit code 0
```

So currently the app is up and the message we get reflects that.

Now let's make our code to fall into the else statement to test the email notification, so a small code change to the if statement:

```
monitor-website.py x
7
8 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
9
10 # if response.status_code == 200:
11 if False:
12     print("Application is running successfully!")
13 else:
14     print("Application Down. Fix it!")
15     # send notification email to support
16     with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
17         smtp.starttls()
18         smtp.ehlo()
19         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
20         msg = "Subject: SITE DOWN\nFix the issue! Restart the application."
21         smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
22 You, 4 minutes ago • Uncommitted changes
Run monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonPr
Application Down. Fix it!
Process finished with exit code 0
```

So log is as expected, now let's check the email:

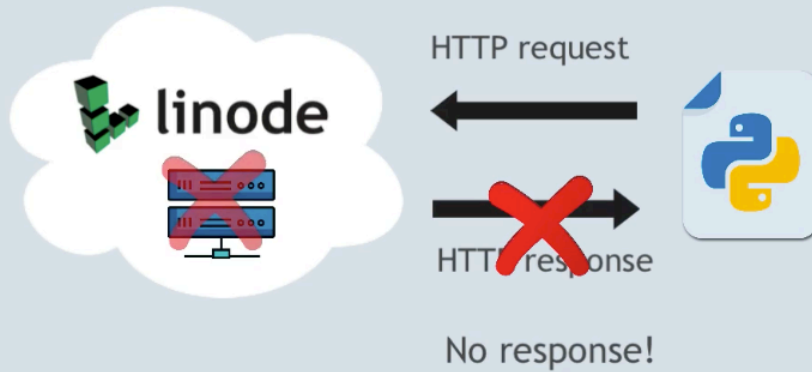


And there it is! 🤔

Handle connection error

What happens if the server doesn't return a response?

Exercise: Handle Connection Errors



If that happens we get an exception at line 9

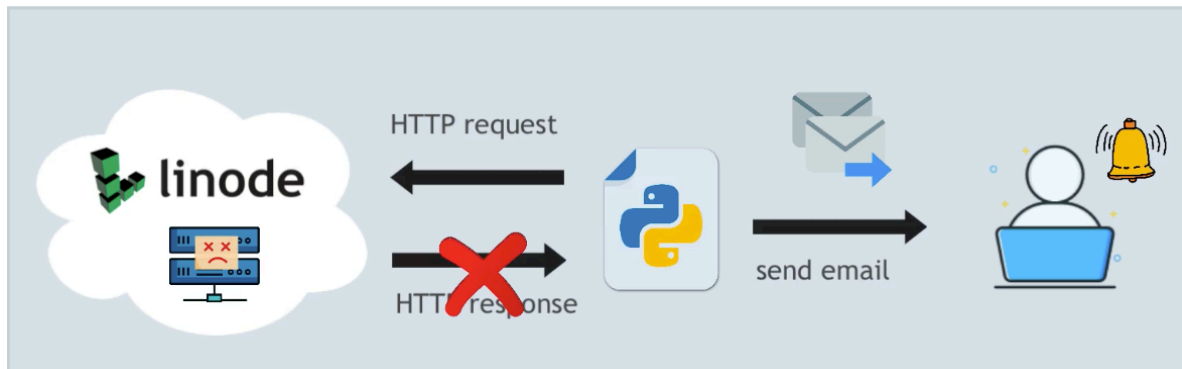
```
3 import os
4
5 EMAIL_ADDRESS = os.environ.get('EMAIL_ADDRESS')
6 EMAIL_PASSWORD = os.environ.get('EMAIL_PASSWORD')
7
8
9 response = requests.get('http://172-104-252-104.ip.linodeusercontent.com:8080/')
10 if response.status_code == 200:
11     print('Application is running successfully!')
12 else:
13     print('Application Down')
14     # send email to me
15     with smtplib.SMTP('smtp.linode.com') as smtp:
16         smtp.starttls()
```

This line will throw an exception

Exception

- when Python script encounters a situation that it cannot cope with, it raises (or throws) an exception.

And in this case we also want an email, as something is wrong with the server (line 10 deals with problems with the application but sometimes the problem is the server)



So we need to do defensive programming and use try/except and Catch the error

Exception

- when Python script encounters a situation that it cannot cope with, it raises (or throws) an exception.
- use try/except in such cases

Let's test the server error by stopping the container from Linode and we will see how currently the python script breaks because we aren't handling this error to fail gracefully.

-> docker stop 52527a5428e8

```
root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
NAMES
52527a5428e8   nginx    "/docker-entrypoint..." 3 hours ago   Up 3 hours
80/tcp        great_elbakyan
root@localhost:~# docker stop 52527a5428e8
52527a5428e8
root@localhost:~#
```

And I run the python script and get connection error due to line 8 below failing:

```
monitor-website.py x
4
5 EMAIL_ADDRESS = os.environ.get("EMAIL_ADDRESS")
6 EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
7
8 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
9
10 if response.status_code == 200:
11     print("Application is running successfully!") Caballero-Pardo, Today • Monitor app access is succe
12 else:
13
Run monitor-website x
return session.request(method=method, url=url, **kwargs)
#####
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/requests/ses
resp = self.send(prepare, **send_kwargs)
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/requests/ses
r = adapter.send(request, **kwargs)
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/requests/adap
raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', por
Process finished with exit code 1
```

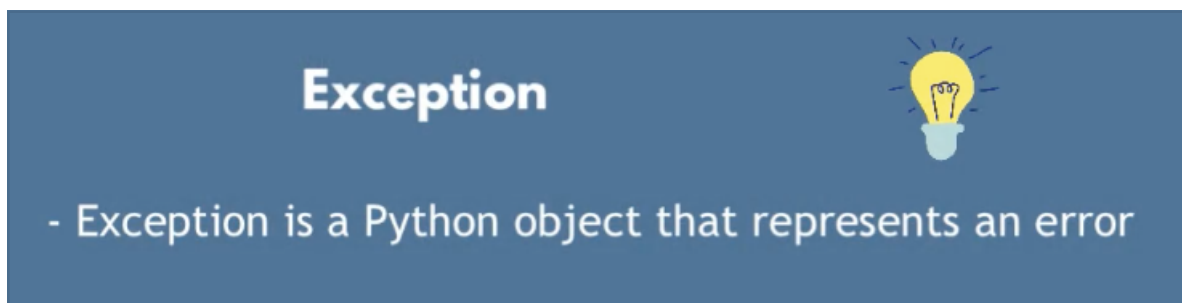
So now we add the try/except syntax

```
8 try:
9     response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
10     if response.status_code == 200:
11         print("Application is running successfully!")
12     else:
13         print("Application Down. Fix it!")
14         # send notification email to support
15         with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
16             smtp.starttls()
17             smtp.ehlo()
18             smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
19             msg = "Subject: SITE DOWN\nFix the issue! Restart the application."
20             smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
21 except Exception as ex: You, Moments ago • Uncommitted changes
```

And for now let's print the exception:


```
monitor-website.py x
8 try:
9     response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
10    if response.status_code == 200:
11        print("Application is running successfully!")
12    else:
13        print("Application Down. Fix it!")
14        # send notification email to support
15        with smtplib.SMTP( host="smtp.gmail.com", port=587) as smtp:
16            smtp.starttls()
17            smtp.ehlo()
18            smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
19            msg = "Subject: SITE DOWN\nFix the issue! Restart the application."
20            smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
21    except Exception as ex:
22        print(f"Connection error happened: {ex}")
23
Run monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
Connection error happened: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', port=8080):
Process finished with exit code 0
```

Exception is a class



Now we can create the email notification:

```
monitor-website.py x
15 with smtplib.SMTP( host="smtp.gmail.com", port=587) as smtp:
16     smtp.ehlo()
17     smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
18     msg = f"Subject: SITE DOWN\nApplication returned {response.status_code}"
19     smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
20
21 except Exception as ex:
22     print(f"Connection error happened: {ex}")
23     # send notification email to support
24     with smtplib.SMTP( host="smtp.gmail.com", port=587) as smtp:
25         smtp.starttls()
26         smtp.ehlo()
27         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
28         msg = "Subject: SITE DOWN\nApplication not accesible at all."
29         smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, msg)
30
```

And we can refactor the code because we are repeating the same code to send email to support so we can make it cleaner:

```
monitor-website.py x
7
8 def send_notification(email_msg): 2+ usages new *
9     with smtplib.SMTP(host="smtp.gmail.com", port=587) as smtp:
10         smtp.starttls()
11         smtp.ehlo()
12         smtp.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
13         message = f"Subject: SITE DOWN\n{email_msg}"
14         smtp.sendmail(EMAIL_ADDRESS, EMAIL_ADDRESS, message)
15
16     try:
17         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
18         if response.status_code == 200:
19             print("Application is running successfully!")
20         else:
21             print("Application Down. Fix it!")
22             # send notification email to support
23             msg = f"Application returned {response.status_code}"
24             send_notification(msg) You, Moments ago • Uncommitted changes
25     except Exception as ex:
26         print(f"Connection error happened: {ex}")
27         # send notification email to support
28         msg = "Application not accessible at all."
29         send_notification(msg)
30
```

And now let's test the code:

```
monitor-website.py x
15
16     try:
17         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
18         if response.status_code == 200:
19             print("Application is running successfully!")
20         else:
21             print("Application Down. Fix it!")
22             # send notification email to support
23             msg = f"Application returned {response.status_code}"
24             send_notification(msg) You, Moments ago • Uncommitted changes
25     except Exception as ex:
26         print(f"Connection error happened: {ex}")
27         # send notification email to support
28         msg = "Application not accessible at all."
29         send_notification(msg)
30
```

Run monitor-website x

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_with_python/monitor-website.py
Connection error happened: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', port=8080): Max retries exceeded with url: / (Caused by OSError: [Errno 111] Connection refused)

 Primary

 Updates

   me

SITE DOWN - Application not accessible at all.

SITE DOWN 

Inbox x



astridcaballero@gmail.com

to bcc: me ▼

Application not accessible at all.

And it works! ❤️